

Week 3 - Day 12a Supplementary: More Sorting!

In this notebook we are exploring common sorting algorithms seen in lecture, like **selection sort**, **insertion sort**, **bubble sort** in more detail. We are analysing their time complexities, implement variants of them and are even going to see some new algorithms like ****

Exercise 1: Time Complexity of Selection Sort []

In the main exercises we have been implementing **selection sort** both iteratively and recursively. What is the time complexity of the selection sort algorithm in the **best** and **worst** cases? Explain your solution. Feel free to use your implementation from the main exercises, and try running it on different inputs to see whether it is faster on certain inputs.

1.1 What is the **best case** time complexity? Please explain your solution.

1.2 What is the **worst case** time complexity? Please explain your solution.

Please ask a TA to check your solution before moving on!

Exercise 2: Bubble Sort []

In this exercise, you will implement the **bubble sort** algorithm - a simple sorting algorithm that works by repeatedly stepping through the list, comparing each pair of adjacent items, and swapping them if they are in the wrong order. The pass through the list is repeated until no more swaps are needed.

Here's how the algorithm works:

1. Compare the first and second elements, and swap them if the first is greater than the second.
2. Compare the second and third elements, and swap them if the second is greater than the third.
3. Continue this process until you reach the end of the list.
4. Repeat the above steps until you make a pass through the list without having to make any swaps.

2.1 Implement a function called `bubble_sort` that takes a list and sort the list **in-place**. This means, that your function should create a new list and should also not return anything. Also, do not use the built-in Python `sort` function! For example,

```
lst = [3,2,1]
sort(lst) == None # But `lst` is now [1,2,3]

# TODO: Your code here
def bubble_sort(lst):
    pass
```

```
# Use this code to test your solutions
lst = [3,2,1,-1,0]
expected = sorted(lst)
bubble_sort(lst)
assert lst == expected

lst = [-100, 9, -1, 3, 1, 0, -3, 2, 5, 5, 5, 5, 5, 1, 100, -100]
expected = sorted(lst)
bubble_sort(lst)
assert lst == expected

lst = [6, 5, 4, 3, 2, 1, 0, -1, -2, -3, -4, -5, -6]
expected = sorted(lst)
bubble_sort(lst)
assert lst == expected

print("All tests passed. ☐")

The expected value is [-3, 0, 1, 2, 3, 5], but the actual value is [3,
1, 0, -3, 2, 5]
The expected value is [-100, -100, -3, -1, 0, 1, 1, 2, 3, 5, 5, 5, 5,
5, 9, 100], but the actual value is [-100, 9, -1, 3, 1, 0, -3, 2, 5,
5, 5, 5, 5, 1, 100, -100]
The expected value is [-6, -5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5, 6],
but the actual value is [6, 5, 4, 3, 2, 1, 0, -1, -2, -3, -4, -5, -6]
```

2.2 What is the time complexity of the bubble sort algorithm in the **best case**? Please explain your answer?

2.2 What is the time complexity of the bubble sort algorithm in the **worst case**? Please explain your answer?

Please ask a TA to check your solution before moving on!

Exercise 3: Reverse Insertion Sort [☐☐☐]

Implement **insertion sort** that sorts an input list `lst` in descending order. Again, please write the algorithm in place, i.e. do not create a new list for your sorting algorithm. Your function should return nothing.

```
# TODO: Your code here
def reverse_insertion_sort(lst):
    pass

# Use this code cell to test your solution
lst = [3, 1, 0, -3, 2, 5]
expected = list(reversed(sorted(lst)))
reverse_insertion_sort(lst) # in-place sorting
assert lst == expected
```

```

lst = [-100, 9, -1, 3, 1, 0, -3, 2, 5, 5, 5, 5, 5, 1, 100, -100]
expected = list(reversed(sorted(lst)))
reverse_insertion_sort(lst) # in-place sorting
assert lst == expected

```

```

lst = [1, 2, 3, 4, 5]
expected = list(reversed(sorted(lst)))
reverse_insertion_sort(lst) # in-place sorting
assert lst == expected

```

```

print("All tests passed. ☑")

```

The expected value is [5, 3, 2, 1, 0, -3], but the actual value is [3, 1, 0, -3, 2, 5]

The expected value is [100, 9, 5, 5, 5, 5, 5, 3, 2, 1, 1, 0, -1, -3, -100, -100], but the actual value is [-100, 9, -1, 3, 1, 0, -3, 2, 5, 5, 5, 5, 5, 1, 100, -100]

The expected value is [5, 4, 3, 2, 1], but the actual value is [1, 2, 3, 4, 5]

Exercise 4: Adaptive Insertion Sort []

Design a Python function for **adaptive insertion sort**. The function should take an already sorted list as input and efficiently handle newly added elements, keeping the list sorted in real-time.

```

# TODO: Your code here
def adaptive_insertion_sort(lst, x):
    pass

# Use this code cell to test your solution
lst = [1, 2, 3, 4, 4, 10, 20]
added_lst = adaptive_insertion_sort(lst, 3)
assert added_lst == [1, 2, 3, 3, 4, 4, 10, 20]

```

```

lst = [1, 1, 5, 5, 100]
added_lst = adaptive_insertion_sort(lst, 500)
assert added_lst == [1, 1, 5, 5, 100, 500]

```

```

lst = [0, 10, 20, 40, 40]
added_lst = adaptive_insertion_sort(lst, 0)
assert added_lst == [0, 0, 10, 20, 40, 40]

```

```

print("All tests passed. ☑")

```

The expected value is [1, 2, 3, 3, 4, 4, 10, 20], but the actual value is [1, 2, 3, 4, 4, 10, 20]

The expected value is [1, 1, 5, 5, 100, 500], but the actual value is [1, 1, 5, 5, 100]

The expected value is [0, 0, 10, 20, 40, 40], but the actual value is [0, 10, 20, 40, 40]

Exercise 5: Counting Operations [] [] []

5.1 Write a function that takes an unsorted list of positive integers as input and **returns the number of comparisons** performed during the **insertion sort** algorithm. By the **number of comparisons**, we mean that you should count the number of times that the algorithm compares two elements of the list.

For example:

The input list is [10, 2, 8, 6, 7, 3].

1. Start from the second element 2. It is smaller than 10, so we swap 2 and 10. This is 1 comparison and 1 swap. The list becomes [2, 10, 8, 6, 7, 3].
2. Then consider 8. It is smaller than 10 but greater than 2, so we swap 8 and 10. This is 2 comparisons (one with 10 and one with 2) and 1 swap. The list becomes [2, 8, 10, 6, 7, 3].
3. Next, consider 6. It is smaller than 10, and also smaller than 8, but not 2, so we swap 6 with both 10 and 8. This is 3 comparisons (with 10, 8, and 2) and 2 swaps. The list becomes [2, 6, 8, 10, 7, 3].
4. Now consider 7. It is smaller than 10, and smaller than 8, but not 6, so we swap 7 with both 10 and 8. This is another 3 comparisons (with 10, 8, and 6) and 2 swaps. The list becomes [2, 6, 7, 8, 10, 3].
5. Lastly, consider 3. It is smaller than all other numbers, so we swap 3 with 10, 8, 7, and 6. This is 5 comparisons (with 10, 8, 7, 6, and 2) and 4 swaps. The list becomes [2, 3, 6, 7, 8, 10].

In total, there is 14 comparisons.

```
# TODO: Your code here
def count_comparisons(lst):
    pass

# Test cases
L1 = [10, 2, 8, 6, 7, 3]
assert count_comparisons(L1) == 14

L2 = [1, 2, 3, 4, 5] # already sorted
assert count_comparisons(L2) == 4

L3 = [5, 4, 3, 2, 1] # reverse sorted
assert count_comparisons(L3) == 10

L5 = [1] # single element
assert count_comparisons(L5) == 0

print("All tests passed. [ ]")
```

The expected value is 14, but the actual value is -1
The expected value is 10, but the actual value is -1
The expected value is 4, but the actual value is -1

The expected value is 0, but the actual value is -1
The expected value is 10, but the actual value is -1
The expected value is 10, but the actual value is -1
The expected value is 0, but the actual value is -1
The expected value is 0, but the actual value is -1

5.1 Write a function that takes an unsorted list of positive integers as input and **returns the number of swaps** performed during the **insertion sort** algorithm. By the **number of swaps**, we mean that you should count the number of times that the algorithm swaps two elements of the list to do the insertion.

For example:

The input list is [10, 2, 8, 6, 7, 3].

1. Start from the second element 2. It is smaller than 10, so we swap 2 and 10. This is 1 comparison and 1 swap. The list becomes [2, 10, 8, 6, 7, 3].
2. Then consider 8. It is smaller than 10 but greater than 2, so we swap 8 and 10. This is 2 comparisons (one with 10 and one with 2) and 1 swap. The list becomes [2, 8, 10, 6, 7, 3].
3. Next, consider 6. It is smaller than 10, and also smaller than 8, but not 2, so we swap 6 with both 10 and 8. This is 3 comparisons (with 10, 8, and 2) and 2 swaps. The list becomes [2, 6, 8, 10, 7, 3].
4. Now consider 7. It is smaller than 10, and smaller than 8, but not 6, so we swap 7 with both 10 and 8. This is another 3 comparisons (with 10, 8, and 6) and 2 swaps. The list becomes [2, 6, 7, 8, 10, 3].
5. Lastly, consider 3. It is smaller than all other numbers, so we swap 3 with 10, 8, 7, and 6. This is 5 comparisons (with 10, 8, 7, 6, and 2) and 4 swaps. The list becomes [2, 3, 6, 7, 8, 10].

In total, there is 10 swaps.

```
# TODO: Your code here
def count_swaps(lst):
    pass

# Use this cell to test your solution
L1 = [10, 2, 8, 6, 7, 3]
assert count_swaps(L1) == 10

L2 = [1, 2, 3, 4, 5] # already sorted
assert count_swaps(L2) == 0

L3 = [5, 4, 3, 2, 1] # reverse sorted
assert count_swaps(L3) == 10

L5 = [1] # single element
assert count_swaps(L5) == 0
```

```
print("All tests passed. ☑")
```

Exercise 6: Cocktail Sort [] []

Cocktail sort is a variation of bubble sort. The algorithm differs from bubble sort in that it sorts in both directions on each pass through the list.

Here's how the algorithm works:

1. Go through the list from beginning to end, swapping adjacent elements if they are in the wrong order.
2. Now go through the list from end to beginning, doing the same thing.
3. Repeat steps 1-2 until the entire list is sorted.

6.1 Write a function `cocktail_sort` that takes a list of numbers `lst` as input and returns a new list that contains the same numbers, sorted in ascending order using cocktail sort. Implement the algorithm in-place, i.e. do not create a new list but modify the input `lst` directly.

```
# TODO: Your code here
def cocktail_sort(lst):
    pass

# Use this cell to test your solution
lst1 = [4, 3, 2, 1]
cocktail_sort(lst1)
assert lst1 == [1, 2, 3, 4]

lst2 = [5, -1, 6, 2, -3]
cocktail_sort(lst2)
assert lst2 == [-3, -1, 2, 5, 6]

lst3 = [1, 2, 3, 4, 3]
cocktail_sort(lst3)
assert lst3 == [1, 2, 3, 3, 4]

print("All tests passed. ☑")
```

The expected value is [1, 2, 3, 4], but the actual value is [4, 3, 2, 1]

The expected value is [-3, -1, 2, 5, 6], but the actual value is [5, -1, 6, 2, -3]

The expected value is [1, 2, 3, 3, 4], but the actual value is [1, 2, 3, 4, 3]

6.2 What is the time complexity of the cocktail sort algorithm in the **best case**? Please explain your answer.

6.3 What is the time complexity of the cocktail sort algorithm in the **worst case**? Please explain your answer.

Exercise 7: "I Can't Believe It Can Sort" Sorting Algorithm

Consider the following [sorting algorithm](#) by Stanley P. Y. Fung that does **in-place** sorting of a list of numbers.

```
def i_cant_believe_it_can_sort(lst):
    n = len(lst)
    for i in range(n):
        for j in range(n):
            if lst[i] < lst[j]:
                # Swap two elements
                lst[i], lst[j] = lst[j], lst[i]
```

This algorithm looks weird on multiple levels:

- Why are we double-looping over the same elements? (e.g. both `i` and `j` can be 0)
- Why are we swapping locations when `lst[i]` is **smaller** than `lst[j]`??

It just seems like we are incorrectly implementing insertion Sort... **But it works!**

```
L = [3, 1, 0, -3, 2, 5]
expected = sorted(L)
i_cant_believe_it_can_sort(L)
check_test(L, expected)
```

```
L = [-1, 3, 1, 0, -3, 2, 5, 5, 5, 5, 5, 1, 100, -100]
expected = sorted(L)
i_cant_believe_it_can_sort(L)
check_test(L, expected)
```

```
L = [-100, 9, -1, 3, 1, 0, -3, 2, 5, 5, 5, 5, 5, 1, 100, -100]
expected = sorted(L)
i_cant_believe_it_can_sort(L)
check_test(L, expected)
```

```
L = [6, 5, 4, 3, 2, 1, 0, -1, -2, -3, -4, -5, -6]
expected = sorted(L)
i_cant_believe_it_can_sort(L)
check_test(L, expected)
```

Congratulations, the test case passed!
Congratulations, the test case passed!
Congratulations, the test case passed!
Congratulations, the test case passed!

Bonus: Can you show/prove that this seemingly wrong sorting algorithm is actually correct?

Hint: For more details read the paper link above if you are curious.

Congratulations 🎉

Exercise 8: Finding the Median of Two Sorted Lists [] [] [] [] []

Write a function that takes two equal-length sorted lists and outputs the median of the two lists. Provide an $O(n)$ time solution.

8.1 Implement the `medianOfTwo` function.

```
# TODO: Implement the function to find the median of two sorted lists
def medianOfTwo(lst1, lst2):
    pass # remove this line and add your code

# Test case

# This function is for testing, do not modify
def check(actual, expected):
    if expected != actual:
        print(f"Failed. Function should return the value {expected},
it is returning the value {actual}")
    else:
        print("Congratulations, the test case passed!")

check(medianOfTwo([1, 4, 5, 9, 10, 12], [2, 3, 6, 7, 8, 11, 13]), 7)

Failed. Function should return the value 7, it is returning the value
None
```

8.2 (Very hard) Can you solve it in $O(\log n)$ time?

```
# TODO: Implement the function to find the median of two sorted lists
in  $O(\log n)$  time
def medianOfTwoFast(lst1, lst2):
    pass # remove this line and add your code

# Test case
check(medianOfTwoFast([1, 4, 5, 9, 10, 12, 14, 15], [2, 3, 6, 7, 8,
11, 13, 16]), 8.5)
```

Congratulations 🎉